Name Of Course	**Bachelor Of Computer Application (BCA)**
Semester	**4TH**
Name Of Subject	**Programming in JAVA**
Subject Code	**BCA-401**
Marks	**75 Marks**
Lectures	**50 Lectures**

Unit I

1. Introduction and Overview of Java

- **1.1 Java History**
- **1.2 Java Features**
- **1.3 How Java Differ from C and C++**
- **1.4 JVM**
- **1.5 Java Environment**
- **1.6 Java Programming Structure**
- **1.7 Java Tokens, Variables, Constants and Data types**
- **1.8 Control Structure**
  - **1.8.1 Conditional Statements**
  - **1.8.2 Looping Statements**
  - **1.8.3 Jumping Statements**
- **1.9 Arrays**

1.1 Java History :

Here's a concise explanation of Java's history :

1. Origin and Development :

Java was developed by **James Gosling** and his team at Sun Microsystems in **1991**. Originally called **"Oak"**, it was intended for programming consumer electronics like TVs and set-top boxes. The name was later changed to "Java" in 1995.

2. Platform Independence:

Java was designed to be **platform-independent**, meaning programs written in Java could run on any device with a **Java Virtual Machine (JVM)**. This was achieved through the "Write Once, Run Anywhere" (WORA) philosophy.

3. Public Launch :

Java's first official version, **Java 1.0**, was released in **1995**. It gained popularity for its simplicity, reliability, and portability.

4. Advancements:

Over the years, Java has evolved with multiple versions, adding new features to improve security, performance, and usability. It is now managed by **Oracle Corporation** after acquiring Sun Microsystems in **2010**.

5. Applications :

Java is widely used for developing web applications, mobile apps (Android), enterprise software, and more, solidifying its place as one of the most popular programming languages in the world.

1.2 Java Features :

Here's a concise explanation of Java features :

1. **Platform Independence:**

   Java follows the "Write Once, Run Anywhere" (WORA) principle. Programs written in Java can run on any device with a **Java Virtual Machine (JVM)**, making it platform-independent.

2. **Object-Oriented:**

   Java is based on **object-oriented programming (OOP)** principles, which improve modularity, code reusability, and scalability. Key concepts include inheritance, encapsulation, and polymorphism.

3. **Simple and Easy to Learn:**

   Java has a clean syntax and eliminates complex features like pointers and operator overloading, making it beginner-friendly.

4. **Secure:**

   Java provides a secure environment for code execution with features like the **Java Sandbox** and runtime checking, which help protect systems from malicious code.

5. **Robust and Reliable:**

   With strong memory management, exception handling, and garbage collection, Java ensures programs are stable and less prone to crashes.

These features make Java a versatile and widely-used programming language.

1.3 How Java Differ from C and C++ :

Here's a concise comparison of how Java differs from C and C++ :

1. **Platform Independence:**

   Java:

   Platform-independent due to the **Java Virtual Machine (JVM)**. Programs can run on any system with JVM.

   C/C++:

   Platform-dependent; compiled code is specific to the target operating system and hardware.

2. **Object-Oriented Programming (OOP):**

   Java:

   Fully object-oriented, with everything (except primitive types) being treated as objects.

   C++:

   Supports both procedural and object-oriented programming.

   C:

   Procedural language with no object-oriented features.

3. **Memory Management:**

   Java:

   Automatic **Garbage Collection** handles memory management.

   C/C++:

   Manual memory management using `malloc`/`free` in C or `new`/`delete` in C++.

4. **Pointers:**

   Java:

   Does not support pointers explicitly for security and simplicity.

   C/C++:

   Provides full support for pointers, allowing direct memory access.

5. Compilation and Execution:

Java:

Compiles into **bytecode**, which is executed by the JVM.

C/C++:

Compiles directly into machine code, making it faster but less flexible across platforms.

These differences highlight Java's focus on portability, simplicity, and security compared to C and C++.

1.4 JVM:

Here's a concise explanation of the Java Virtual Machine (JVM):

1. Definition:

The **Java Virtual Machine (JVM)** is a virtual runtime environment that allows Java programs to execute. It acts as a mediator between Java bytecode and the underlying operating system.

2. Functionality:

Converts **Java bytecode** (compiled from `.java` files) into machine code that the computer can understand and execute.

Ensures Java's **platform independence** by providing an environment that is the same across different systems.

3. Key Responsibilities:

Class Loading: Loads compiled `.class` files (bytecode) into memory.

Bytecode Verification: Ensures bytecode is secure and doesn't harm the host system.

Execution: Uses an **Interpreter** or **Just-In-Time (JIT) Compiler** to execute the program efficiently.

Memory Management: Includes features like **Garbage Collection** to automatically manage memory allocation and deallocation.

4. Components:

Class Loader: Loads classes into memory.

Execution Engine: Executes bytecode instructions.

Runtime Data Areas: Includes **Heap**, **Stack**, and **Method Area**, which manage runtime data storage.

5. Importance:

The JVM ensures Java's **Write Once, Run Anywhere (WORA)** principle, enabling Java programs to run seamlessly across different platforms.

This makes the JVM a crucial part of Java's architecture.

1.5 Java Environment:

Here's a concise explanation of the Java Environment:

1. Definition:

The **Java Environment** is the setup required to write, compile, and run Java programs. It includes tools, libraries, and components necessary for Java application development and execution.

2. Key Components:

Java Development Kit (JDK):

A complete development environment for building Java applications. It includes:

Compiler (javac): Converts source code (`.java`) into bytecode (`.class`).

Debugger: Helps in identifying and fixing errors in programs.

Java Libraries: Predefined classes and methods for faster development.

Java Runtime Environment (JRE):

Provides the libraries, JVM, and other tools needed to run Java applications. It is a subset of the JDK.

Java Virtual Machine (JVM):

The core component of the JRE responsible for executing the Java bytecode, ensuring platform independence.

3. Workflow:

Write:

Developers write Java code in `.java` files using an Integrated Development Environment (IDE) like Eclipse or IntelliJ.

Compile:

The JDK's **javac** compiler translates the source code into bytecode.

Execute:

The JVM executes the bytecode, using the JRE for necessary resources.

4. Importance:

The Java Environment ensures a seamless process for developers to write, test, and deploy applications across multiple platforms.

This modular setup makes Java development efficient and accessible.

1.6 Java Programming Structure:

Here's a concise explanation of the Java Programming Structure:

1. Structure Overview:

A Java program follows a well-defined structure, starting with essential elements like class declarations, methods, and statements. The program typically begins with a `main()` method, which serves as the entry point for execution.

2. Key Components:

Package Declaration (Optional):

Declares the package that the class belongs to. For example:

```
package mypackage;
```

Import Statements (Optional):

Imports necessary classes or packages. For example:

```
import java.util.Scanner;
```

Class Declaration:

Every Java program is encapsulated in a class. For example:

```
public class MyClass {
}
```

Main Method:

The entry point of the program. It is where execution begins.

```
public static void main(String[] args) {
}
```

3. Structure Example:

```
// Package Declaration (Optional)
package mypackage;
```

```
// Import Statements (Optional)
import java.util.Scanner;

// Class Declaration
public class MyClass {
    // Main Method
    public static void main(String[] args) {
        // Statements
        System.out.println("Hello, World!");
    }
}
```

4. Rules:

The file name must match the public class name.
Every statement ends with a semicolon (`;`).
Use proper indentation for readability.

5. Execution Flow:

Write the program in a `.java` file.
Compile it using `javac`.
Run the compiled bytecode with `java`.

This structure ensures consistency and readability in Java programming.

1.7 Java Tokens, Variables, Constants and Data types:

Here's a concise explanation of Java Tokens, Variables, Constants, and Data Types:

1. Java Tokens:

The smallest elements of a Java program that are meaningful to the compiler.
Types of Tokens:

Keywords: Reserved words like `class`, `public`, `if`.
Identifiers: Names for variables, classes, and methods.
Literals: Fixed values like `123`, `"Hello"`.
Operators: Symbols like `+`, `-`, `*`, `/`.
Separators: Symbols like `{`, `}`, `;`.

2. Variables:

Containers for storing data values in a program.
Declaration:

```
int age = 25; // Variable "age" of type int
```

Rules:

Must start with a letter, `$`, or `_`.
Cannot be a keyword.

Types of Variables:

Local: Declared inside a method or block.
Instance: Declared in a class but outside methods.
Static: Declared with the `static` keyword and shared among all instances of a class.

3. Constants:

Fixed values that cannot be changed during execution.

Declared using the `final` keyword:

final double PI = 3.14159;

Constants are usually named in **uppercase** for readability.

4. Data Types:

Define the type of data a variable can hold.

Primitive Data Types:

Numeric: `int`, `float`, `double`, `byte`, `short`, `long`.

Character: `char`.

Boolean: `boolean` (true/false).

Non-Primitive Data Types:

Objects, Strings, Arrays.

Example:

String name = "Java";

Importance:

These elements form the building blocks of Java programming, ensuring proper data management and program execution.

1.8.1 Conditional Statements:

Here's a concise explanation of Conditional Statements in Java:

1. Definition:

Conditional statements allow a program to make decisions and execute specific blocks of code based on conditions. They enable dynamic and logical flow control in a program.

2. Types of Conditional Statements:

1. `if` Statement:

Executes a block of code if the condition is true.

Syntax:

```
if (condition) {
    // Code to execute
}
```

Example:

```
if (age > 18) {
    System.out.println("You are an adult.");
}
```

2. `if-else` Statement:

Executes one block if the condition is true and another if it is false.

Syntax:

```
if (condition) {
    // Code if condition is true
} else {
    // Code if condition is false
}
```

Example:

```
if (marks >= 50) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

3. `if-else if` Ladder:

Checks multiple conditions sequentially.

Syntax:

```
if (condition1) {
    // Code for condition1
} else if (condition2) {
    // Code for condition2
} else {
    // Code if none are true
}
```

Example:

```
if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 75) {
    System.out.println("Grade B");
} else {
    System.out.println("Grade C");
}
```

4. `switch` Statement:

Evaluates a single expression and executes a matching case.

Syntax:

```
switch (expression) {
    case value1:
        // Code for case 1
        break;
    case value2:
        // Code for case 2
        break;
    default:
        // Code if no case matches
}
```

Example:

```
switch (day) {
    case 1:
        System.out.println("Monday");
```

```
            break;
        case 2:
            System.out.println("Tuesday");
            break;
        default:
            System.out.println("Invalid day");
    }
```

3. Importance:

Conditional statements are fundamental for decision-making in programs, enabling dynamic and flexible code execution.

1.8.2 Looping Statements:

Here's a concise explanation of Looping Statements in Java:

1. Definition:

Looping statements in Java are used to repeatedly execute a block of code as long as a specified condition is true. They help reduce redundancy and improve code efficiency.

2. Types of Looping Statements:

1. `for` Loop:

Used when the number of iterations is known beforehand.

Syntax:

```
for (initialization; condition; update) {
    // Code to execute
}
```

Example:

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Iteration " + i);
}
```

2. `while` Loop:

Used when the number of iterations is not fixed, and the condition is checked before entering the loop.

Syntax:

```
while (condition) {
    // Code to execute
}
```

Example:

```
int i = 1;
while (i <= 5) {
    System.out.println("Iteration " + i);
    i++;
}
```

3. `do-while` Loop:

Similar to the `while` loop, but the condition is checked after executing the loop, ensuring the code runs at least once.

Syntax:

```
do {
    // Code to execute
} while (condition);
```

Example:

```
int i = 1;
do {
    System.out.println("Iteration " + i);
    i++;
} while (i <= 5);
```

4. Enhanced `for` Loop (For-Each):

Specifically used for iterating over arrays or collections.

Syntax:

```
for (type variable : array) {
    // Code to execute
}
```

Example:

```
int[] numbers = {1, 2, 3, 4, 5};
for (int num : numbers) {
    System.out.println(num);
}
```

3. Importance:

Looping statements simplify repetitive tasks, making programs more efficient and concise. They are essential for tasks like traversing arrays, reading data, or performing repeated computations.

1.8.3 Jumping Statements:

Here's a concise explanation of Jumping Statements in Java:

1. Definition:

Jumping statements in Java control the flow of execution by transferring control to specific points in the program. They are used to exit loops, skip iterations, or jump to labeled blocks.

2. Types of Jumping Statements:

1. `break` Statement:

Terminates the current loop or switch statement prematurely.

Syntax:

```
break;
```

Example (in a loop):

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
```

```
            break; // Exits the loop when i is 3
        }
        System.out.println(i);
    }
```

Example (in `switch`):

```
    switch (day) {
        case 1:
            System.out.println("Monday");
            break;
        case 2:
            System.out.println("Tuesday");
            break;
        default:
            System.out.println("Other day");
    }
```

2. `continue` Statement:

Skips the remaining code in the current iteration of a loop and proceeds to the next iteration.

Syntax:

```
    continue;
```

Example:

```
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            continue; // Skips iteration when i is 3
        }
        System.out.println(i);
    }
```

3. `return` Statement:

Exits from the current method and optionally returns a value to the calling method.

Syntax:

```
    return; // No value
    return value; // With a value
```

Example:

```
    public int add(int a, int b) {
        return a + b; // Returns the sum to the calling method
    }
```

4. Labeled Statements (Optional Use):

Used with `break` or `continue` to jump to a specific labeled block.

Syntax:

```
    label:
        for (int i = 0; i < 5; i++) {
```

```
            if (i == 3) {
                break label; // Exits the labeled block
            }
        }
```

3. Importance:

Jumping statements enhance control flow, making it easier to handle specific conditions like exiting loops, skipping iterations, or returning results from methods. They are crucial for efficient program logic.

1.9 Arrays:

Here's a concise explanation of Arrays in Java:

1. Definition:

An **array** is a data structure in Java used to store multiple values of the same data type in a contiguous memory location. Arrays are fixed in size and allow indexed access to their elements.

2. Features:

Fixed Size: The size of an array is declared at the time of creation and cannot be changed.

Indexed: Elements are accessed using zero-based indexing (`arr[0]` is the first element).

Homogeneous: An array can only store elements of the same data type.

Efficient: Provides faster access to elements using indices.

3. Declaration and Initialization:

Declaration:

```
dataType[] arrayName; // Preferred
dataType arrayName[]; // Also valid
```

Initialization:

```
int[] numbers = new int[5]; // Array of size 5
int[] values = {1, 2, 3, 4, 5}; // Direct initialization
```

4. Accessing Elements:

Elements are accessed using their indices:

```
int[] numbers = {10, 20, 30};
System.out.println(numbers[1]); // Outputs: 20
```

5. Types of Arrays:

1. One-Dimensional Array:

A single row of elements.

Example:

```
int[] arr = {1, 2, 3, 4};
```

2. Multi-Dimensional Array:

An array of arrays, commonly used for matrices.

Example:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(matrix[1][2]); // Outputs: 6
```

6. Advantages:

Efficient storage for multiple elements.

Easy to iterate using loops (e.g., `for` or enhanced `for`).

Allows quick access and modification using indices.

7. Example Program:

```
public class ArrayExample {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40};
        for (int num : numbers) {
            System.out.println(num);
        }
    }
}
```

8. Importance:

Arrays are fundamental for managing collections of data, enabling efficient storage, access, and manipulation in Java programs.

Unit II

2. Classes, Objects and Methods

2.1 Defining Class, Fields Declaration, Methods Declaration, Creating Objects
2.2 Visibility controls
2.3 Use of 'this' Keyword
2.4 Method Parameters and Method Overloading
2.5 Constructor and Constructor Overloading
2.6 Static Members
2.7 Finializer Method
2.8 Inheritance and It's Types
2.9 Method Overriding
2.10 Final Variable, Method and Final Class

2.1 Defining Class, Fields Declaration, Methods Declaration, Creating Objects:

Here's a concise explanation of Defining Class, Fields Declaration, Methods Declaration, and Creating Objects in Java:

1. Defining a Class:

A class is a blueprint for creating objects. It defines the properties (fields) and behaviors (methods) of objects.

Syntax:

```
class ClassName {
    // Fields and Methods
}
```

Example:

```
class Car {
    String model;
    int year;
}
```

2. Fields Declaration:

Fields (or attributes) are variables that hold the state of a class.

Syntax:

```
dataType fieldName;
```

Example:

```
class Car {
    String model; // Field for car model
    int year;     // Field for car manufacturing year
}
```

3. Methods Declaration:

Methods define the behavior or actions of a class.

Syntax:

```
returnType methodName(parameters) {
    // Method body
}
```

Example:

```
class Car {
    String model;
    int year;

    // Method to display car details
    void displayDetails() {
        System.out.println("Model: " + model + ", Year: " + year);
    }
}
```

4. Creating Objects:

Objects are instances of a class, created using the `new` keyword.

Syntax:

```
ClassName objectName = new ClassName();
```

Example:

```
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();  // Create an object of Car
        myCar.model = "Tesla"; // Set field values
        myCar.year = 2023;
        myCar.displayDetails(); // Call method
    }
}
```

5. Importance:

Defining classes, declaring fields, creating methods, and instantiating objects are fundamental steps in **Object-Oriented Programming (OOP)**. They allow the creation of modular, reusable, and organized code.

2.2 Visibility controls:

Here's a concise explanation of Visibility Controls in Java:

1. Definition:

Visibility controls in Java manage the accessibility of classes, methods, and variables to other parts of a program. This is achieved using **access modifiers**, ensuring proper encapsulation and security of the code.

2. Access Modifiers:

1. Public:

Makes the member accessible from **anywhere** in the program.

Example:

```
public class MyClass {
    public void display() {
        System.out.println("Public method");
    }
}
```

2. Private:

Restricts access to the member **within the same class only**.

Example:

```
class MyClass {
    private int data = 10;
    private void display() {
        System.out.println("Private method");
    }
}
```

3. Protected:

Allows access to members in the **same package** and subclasses in **different packages**.

Example:

```
class MyClass {
    protected int data = 20;
}
```

4. Default (No Modifier):

Members are accessible **within the same package** only.

Example:

```
class MyClass {
    int data = 30; // Default access
}
```

3. Summary of Access Modifiers:

Modifier	Same Class	Same Package	Subclass (Diferent Package)	Everywhere
public	✔	✔	✔	✔
protected	✔	✔	✔	✘
default	✔	✔	✘	✘
Private	✔	✘	✘	✘

4. Importance:

Visibility controls help:

Protect sensitive data using `private`.

Allow controlled access via `public` and `protected`.

Facilitate **encapsulation**, a key principle of Object-Oriented Programming (OOP).

This ensures secure, modular, and maintainable code.

2.3 Use of 'this' Keyword:

Here's a concise explanation of the `this` Keyword in Java:

1. Definition:

The **`this` keyword** is a reference variable in Java that refers to the **current object** of a class. It is used to resolve naming conflicts, access class members, and pass the current object as a parameter.

2. Uses of `this` Keyword:

1. Referring to Instance Variables:

When a local variable name and an instance variable name are the same, `this` helps distinguish between them.

Example:

```
class Example {
    int num;

    void setNum(int num) {
        this.num = num; // Resolves naming conflict
    }
}
```

2. Calling Another Constructor (Constructor Chaining):

`this()` can be used to call another constructor of the same class.

Example:

```
class Example {
    Example() {
        System.out.println("Default Constructor");
    }
    Example(int x) {
        this(); // Calls the default constructor
        System.out.println("Parameterized Constructor: " + x);
    }
}
```

3. Passing the Current Object as an Argument:

The current object can be passed as an argument to a method or constructor using `this`.

Example:

```
class Example {
    void display(Example obj) {
        System.out.println("Method called with current object");
    }
    void call() {
        display(this); // Passes current object
    }
}
```

4. Returning the Current Object:

`this` can be used to return the current class instance.

Example:

```
class Example {
    Example getObject() {
        return this; // Returns current object
    }
}
```

3. Importance:

Resolves ambiguity between instance and local variables.

Enables **constructor chaining** for efficient initialization.

Facilitates **method calls** with the current object.

Ensures better code clarity and maintainability.

The `this` keyword is essential for working with objects in Java and adhering to the principles of Object Oriented Programming (OOP).

2.4 Method Parameters and Method Overloading:

Here's a concise explanation of Method Parameters and Method Overloading in Java:

1. Method Parameters:

Definition:

Parameters are variables passed to a method to provide input data when the method is called.

Types of Parameters:

Formal Parameters: Defined in the method declaration.

Example:

```
void display(String message) {
    System.out.println(message); // 'message' is the formal parameter
}
```

Actual Parameters (Arguments): The values passed to the method during the method call.

Example:

```
display("Hello, World!"); // "Hello, World!" is the actual parameter
```

Pass by Value: Java passes parameters by value, meaning the method gets a copy of the arguments.

2. Method Overloading:

Definition:

Method overloading allows multiple methods in the same class to have the same name but differ in their **parameter list** (number, type, or order of parameters).

Purpose:

Provides flexibility to perform similar operations with different inputs.

Rules for Method Overloading:

1. Methods must have the **same name** but different parameter lists.

2. The return type can be different but is not considered for overloading.

Example:

```
class Calculator {
    // Method to add two integers
    int add(int a, int b) {
        return a + b;
    }

    // Overloaded method to add three integers
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Overloaded method to add two doubles
    double add(double a, double b) {
        return a + b;
    }
}
```

3. Example Program:

```
public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));        // Calls add(int, int)
        System.out.println(calc.add(2, 3, 4));     // Calls add(int, int, int)
        System.out.println(calc.add(2.5, 3.5));    // Calls add(double, double)
    }
}
```

4. Importance:

Method Parameters provide a way to pass dynamic data to methods for processing.

Method Overloading enhances code readability and reusability by allowing methods with similar functionality to coexist.

These features ensure flexibility and adherence to the principles of Object-Oriented Programming (OOP).

2.5 Constructor and Constructor Overloading:

Here's a concise explanation of Constructor and Constructor Overloading in Java:

1. Constructor:

Definition:

A constructor is a special method used to initialize objects when they are created.

Characteristics:

It has the **same name** as the class.

No return type, not even `void`.

Automatically called when an object is instantiated.

Example:

```
class Person {
    String name;

    // Constructor
    Person(String name) {
        this.name = name; // Initializes the object
    }

    void display() {
        System.out.println("Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Person person = new Person("John"); // Constructor called
        person.display();
    }
}
```

2. Types of Constructors:

1. Default Constructor:

Automatically provided by Java if no constructor is defined.

Initializes objects with default values.

Example:

```
class Example {
    Example() {
        System.out.println("Default Constructor");
    }
}
```

2. Parameterized Constructor:

Takes arguments to initialize objects with specific values.

Example:

```
class Example {
    Example(int value) {
        System.out.println("Value: " + value);
    }
}
```

3. Constructor Overloading:

Definition:

Constructor overloading is the process of defining multiple constructors in a class with different parameter lists.

Purpose:

Provides flexibility to create objects in different ways.

Example:

```
class Person {
    String name;
    int age;

    // Default constructor
    Person() {
        name = "Unknown";
        age = 0;
    }

    // Parameterized constructor
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Person person1 = new Person(); // Calls default constructor
        Person person2 = new Person("Alice", 25); // Calls parameterized
      constructor
        person1.display();
        person2.display();
    }
}
```

4. Importance:

Constructors ensure proper initialization of objects.

Constructor Overloading allows flexibility by providing multiple ways to initialize objects, enhancing code usability and maintainability.

These features are vital in object-oriented programming for creating robust and flexible code.

2.6 Static Members:

Here's a concise explanation of Static Members in Java:

1. Definition:

Static members in Java refer to fields (variables) and methods that are associated with the **class** rather than any specific instance of the class. They are shared among all instances of the class.

2. Static Fields (Variables):

Definition:

Declared using the `static` keyword, these fields are common for all objects of the class.

Purpose:

Useful for storing shared data or constants.

Example:

```
class Example {
    static int count = 0; // Static variable

    Example() {
        count++; // Increment on object creation
    }
}
```

3. Static Methods:

Definition:

Declared using the `static` keyword, these methods can be called without creating an object of the class.

Restrictions:

Can only access **static variables** and call **static methods** directly.

Cannot use `this` or `super`.

Example:

```
class Example {
    static void displayMessage() {
        System.out.println("This is a static method.");
    }
}

public class Main {
    public static void main(String[] args) {
        Example.displayMessage(); // Call without creating an object
    }
}
```

4. Static Block:

Definition:

A block of code executed once when the class is loaded into memory.

Purpose:

Used to initialize static fields.

Example:

```
class Example {
    static int count;

    static {
        count = 10; // Static block initializes static variable
    }
```

```
        }
```

5. Advantages of Static Members:

Shared memory saves resources.

Can be accessed directly using the class name.

Ideal for utility or helper methods (e.g., `Math.sqrt()`).

6. Example Program:

```
class Example {
        static int count = 0; // Static variable

        Example() {
                count++;
        }

        static void displayCount() { // Static method
                System.out.println("Count: " + count);
        }
}

public class Main {
        public static void main(String[] args) {
                new Example();
                new Example();
                Example.displayCount(); // Outputs: Count: 2
        }
}
```

7. Importance:

Static members enable efficient memory usage and provide a way to share data or methods across all objects of a class. They are commonly used for constants, utility methods, or tracking class-level properties.

2.7 Finializer Method:

Here's a concise explanation of the Finalizer Method in Java:

1. Definition:

The **finalizer method** in Java, defined as `protected void finalize()`, is called by the garbage collector before an object is destroyed. It allows cleanup operations such as releasing resources (e.g., closing files, network connections).

2. Syntax:

```
protected void finalize() throws Throwable {
        // Cleanup code
}
```

3. Features:

Automatic Invocation:

The `finalize` method is automatically called by the garbage collector.

Override Allowed:

You can override it to define custom cleanup behavior for objects.

Rarely Used:

Not commonly used in modern Java due to better alternatives like `try-with-resources`.

Execution Not Guaranteed:

The JVM doesn't guarantee when or if `finalize` will be called.

4. Example:

```
class Example {
    @Override
    protected void finalize() throws Throwable {
        System.out.println("Finalizer called for object: " + this);
    }
}

public class Main {
    public static void main(String[] args) {
        Example obj1 = new Example();
        Example obj2 = new Example();
        obj1 = null; // Eligible for garbage collection
        obj2 = null; // Eligible for garbage collection
        System.gc(); // Request garbage collection
        System.out.println("Garbage collection requested");
    }
}
```

Output:

"Garbage collection requested"

(Possibly) "Finalizer called for object: Example@someHashCode"

5. Limitations:

Unpredictable Timing:

The JVM controls garbage collection, so the `finalize` method might not be called immediately.

Deprecated in Java 9 and Later:

`finalize` has been replaced with better resource management techniques, like `try-with resources` and `AutoCloseable`.

6. Importance:

Historically used for cleanup before object destruction.

Not recommended in modern Java development due to performance issues and unpredictability.

Alternatives like `try-with-resources` are now preferred for managing resources efficiently.

2.8 Inheritance and It's Types:

Here's a concise explanation of Inheritance and its types in Java:

1. Definition of Inheritance:

Inheritance is a feature in Java that allows a class (**child class**) to acquire the properties and methods of another class (**parent class**). It promotes **code reuse**, **modularity**, and supports **Object Oriented Programming (OOP)** principles.

2. Syntax:

Inheritance is implemented using the `extends` keyword:

```
class Parent {
    // Parent class properties and methods
}

class Child extends Parent {
    // Child class inherits from Parent
}
```

3. Types of Inheritance:

1. Single Inheritance:

A child class inherits from one parent class.

Example:

```
class Parent {
    void display() {
        System.out.println("Parent class method");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child class method");
    }
}
```

2. Multilevel Inheritance:

A class inherits from another class, which itself inherits from another class.

Example:

```
class Grandparent {
    void displayGrandparent() {
        System.out.println("Grandparent class method");
    }
}

class Parent extends Grandparent {
    void displayParent() {
        System.out.println("Parent class method");
    }
```

```
}

class Child extends Parent {
    void displayChild() {
        System.out.println("Child class method");
    }
}
```

3. Hierarchical Inheritance:

Multiple child classes inherit from a single parent class.

Example:

```
class Parent {
    void display() {
        System.out.println("Parent class method");
    }
}
class Child1 extends Parent {
    void show() {
        System.out.println("Child1 class method");
    }
}
class Child2 extends Parent {
    void print() {
        System.out.println("Child2 class method");
    }
}
```

4. Unsupported Inheritance Types:

1. Multiple Inheritance:

Java does not support multiple inheritance with classes to avoid ambiguity issues. Instead, it can be achieved using **interfaces**.

5. Importance:

Promotes **code reuse** by reducing redundancy.

Simplifies the structure and hierarchy of a program.

Supports **polymorphism** (e.g., method overriding).

Inheritance is a core principle of Object-Oriented Programming that helps in building modular and maintainable software systems.

2.9 Method Overriding:

Here's a concise explanation of Method Overriding in Java:

1. Definition:

Method Overriding in Java occurs when a subclass provides a **specific implementation** of a method that is already defined in its parent class. The overridden method in the child class has the **same name, return type, and parameters** as the method in the parent class.

2. Key Characteristics:

1. The method must be defined in both the parent and child classes with the **same signature.**
2. The overriding method cannot have a **weaker access modifier** than the method in the parent class.
   For example, if the parent method is `protected`, the child method cannot be `private`.
3. It supports **runtime polymorphism.**
4. The `@Override` annotation is used to ensure the method is correctly overridden.

3. Syntax:

```
class Parent {
    void display() {
        System.out.println("Parent class method");
    }
}

class Child extends Parent {
    @Override
    void display() {
        System.out.println("Child class method");
    }
}
```

4. Example Program:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Dog(); // Upcasting
        myAnimal.sound(); // Calls the overridden method in Dog class
    }
}
```

Output:

```
Dog barks
```

5. Rules for Method Overriding

1. **Same Signature:** The method name, return type, and parameters must match.
2. **Access Modifiers:** The child class method should have an access level equal to or broader than the parent class method.
3. **Static and Final Methods:** These methods cannot be overridden.
4. **Constructor Overriding:** Constructors cannot be overridden.

6. Importance:

Enables **runtime polymorphism**, allowing dynamic method calls based on the object type.
Provides flexibility for a subclass to customize or extend the behavior of a parent class method.
Encourages reusable and maintainable code.

Method Overriding is a fundamental feature of Object-Oriented Programming (OOP) that enhances the design and functionality of Java programs.

2.10 Final Variable, Method and Final Class:

Here's a concise explanation of Final Variable, Final Method, and Final Class in Java:

1. Final Variable:

A **final variable** is a constant; once assigned a value, it cannot be changed.
Declaration: Use the `final` keyword to declare the variable.
Example:

```
class Example {
    final int MAX_VALUE = 100; // Final variable
}
```

Key Points:

Must be initialized either at the time of declaration or in the constructor.
Commonly used for constants.

2. Final Method:

A **final method** cannot be overridden by subclasses.
Purpose: To prevent a method from being modified in derived classes, ensuring consistent behavior.
Example:

```
class Parent {
    final void display() {
        System.out.println("This is a final method.");
    }
}

class Child extends Parent {
    // Cannot override display() method
}
```

Key Points:

Helps maintain the integrity of critical methods.
Allows a class to control how specific methods are implemented.

3. Final Class:

A **final class** cannot be extended (no subclasses can be created).

Purpose: To prevent inheritance, often used for security or immutability.

Example:

```
final class Example {
    void display() {
        System.out.println("This is a final class.");
    }
}

// class Child extends Example { } // Error: Cannot extend final class
```

Key Points:

Ensures the class's implementation cannot be altered through inheritance.

Commonly used in utility classes like `java.lang.String`.

4. Summary Table:

Feature	Usage	Purpose
Final Variable	Prevents the value from being changed.	Used to declare constants.
Final Method	Prevents method overriding	Maintains consistent behavior.
Final Class	Prevents class inheritance.	Ensures class security or immutablity.

5. Importance:

The `final` keyword in Java is critical for:

Enhancing **code stability** and **security**.

Preventing **unintended changes** to variables, methods, or classes.

Supporting **immutability**, a key aspect of reliable and efficient programming.

Unit III

3. Interface, Package and Exception Handling

3.1 Defining and implementing interface

3.2 Package

3.2.1 Create Package

3.2.2 Accessing Package

3.3 Exception

3.3.1 Types of Error

3.3.2 Multiple catch statement

3.3.3 Creating User defined Exception

3.3.4 Finally clause

3.1 Defining and implementing interface:

Here's a concise explanation of Defining and Implementing an Interface in Java:

1. What is an Interface?

An **interface** in Java is a blueprint of a class that contains **abstract methods** (methods without implementation) and **static constants**. It is used to achieve full **abstraction** and **multiple inheritance** in Java.

2. Defining an Interface:

An interface is defined using the `interface` keyword.

It can contain:

Abstract methods (implicitly public and abstract).

Static and final constants.

Example:

```
interface Animal {
    void sound(); // Abstract method
    void eat();   // Abstract method
}
```

3. Implementing an Interface:

A class implements an interface using the `implements` keyword.

The implementing class must provide implementations for all abstract methods in the interface.

Syntax:

```
class ClassName implements InterfaceName {
    // Provide implementations for all methods
}
```

Example:

```
interface Animal {
    void sound();
    void eat();
```

```
        }

        class Dog implements Animal {
            @Override
            public void sound() {
                System.out.println("Dog barks");
            }

            @Override
            public void eat() {
                System.out.println("Dog eats bones");
            }
        }
```

4. Example Program:

```
    interface Shape {
        void draw(); // Abstract method
    }

    class Circle implements Shape {
        @Override
        public void draw() {
            System.out.println("Drawing a Circle");
        }
    }

    class Rectangle implements Shape {
        @Override
        public void draw() {
            System.out.println("Drawing a Rectangle");
        }
    }

    public class Main {
        public static void main(String[] args) {
            Shape circle = new Circle();
            Shape rectangle = new Rectangle();
            circle.draw();
            rectangle.draw();
        }
    }
```

Output:

```
    Drawing a Circle
    Drawing a Rectangle
```

5. Key Features of Interfaces:

1. Supports **multiple inheritance** (a class can implement multiple interfaces).
2. Methods in an interface are implicitly **public** and **abstract**.
3. Fields in an interface are implicitly **public**, **static**, and **final**.
4. Interfaces cannot have constructors or instance variables.

6. Importance:

Promotes **polymorphism** and **loose coupling**.

Allows **standardization** in class design by defining common behavior.

Achieves **multiple inheritance** in Java, which is otherwise not supported with classes.

3.2.1 Create Package:

Here's a concise explanation of Creating a Package in Java:

1. What is a Package?

A **package** in Java is a way to organize classes and interfaces into namespaces, similar to folders in a file directory. It helps in managing and structuring the code effectively.

2. Why Use Packages?

1. To **group related classes** and interfaces.
2. To **avoid name conflicts** by providing unique namespaces.
3. To make it easier to **maintain** and **reuse** code.
4. To provide **access control** using package-private and public access levels.

3. Creating a Package:

1. Define the Package: Use the `package` keyword at the top of the file.

2. Compile the Code: Use the `javac -d` option to specify the directory for package placement.

3. Access the Package: Import the package in other classes using the `import` keyword.

4. Example:

Step 1: Define a Package

Create a file `MyPackageClass.java` with the following content:

```
package mypackage; // Declare the package

public class MyPackageClass {
    public void displayMessage() {
        System.out.println("Hello from the package!");
    }
}
```

Step 2: Compile the Package

```
javac -d . MyPackageClass.java
```

This creates a directory structure like `mypackage/MyPackageClass.class`.

Step 3: Use the Package

Create another file `Main.java` outside the `mypackage` folder:

```
import mypackage.MyPackageClass; // Import the package

public class Main {
    public static void main(String[] args) {
        MyPackageClass obj = new MyPackageClass();
        obj.displayMessage();
    }
}
```

Step 4: Compile and Run

```
javac Main.java
java Main
```

Output:

```
Hello from the package!
```

5. Key Points:

Default Package: If no package is declared, the class belongs to the default package.

Access Control: Classes and members with default access are accessible only within the same package.

Nested Packages: You can create sub-packages, e.g., `mypackage.subpackage`.

6. Importance:

Packages help in organizing large projects effectively.

They prevent class name conflicts.

They improve code readability and reusability.

Built-in Java packages (e.g., `java.util`, `java.io`) simplify programming by providing pre-defined classes and methods.

3.2.2 Accessing Package:

Here's a concise explanation of Accessing Packages in Java:

1. What is Accessing a Package?

Accessing a package in Java refers to importing and using classes, interfaces, or sub-packages from a defined package in your program.

2. Steps to Access a Package:

1. Use `import` Statement:

To access a class or interface from a package, use the `import` keyword.

Syntax:

```
import package_name.ClassName; // Import a specific class
import package_name.*;       // Import all classes in a package
```

2. Fully Qualified Name (Optional):

Classes from a package can also be accessed without `import` by using their **fully qualified**

name.

Example:

```
package_name.ClassName obj = new package_name.ClassName();
```

3. Example of Accessing a Package:

Step 1: Create a Package

Create a file `mypackage/MyClass.java`:

```
package mypackage; // Define the package

public class MyClass {
    public void display() {
        System.out.println("Hello from MyClass in mypackage!");
    }
}
```

Compile it using:

```
javac -d . MyClass.java
```

Step 2: Access the Package

Create a file `Main.java` in the same project:

```
import mypackage.MyClass; // Import the specific class

public class Main {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.display(); // Access the method
    }
}
```

4. Access Control:

1. **Public Members:** Accessible from any class in any package.
2. **Default Members:** Accessible only within the same package.
3. **Protected Members:** Accessible in the same package and through inheritance in other packages.
4. **Private Members:** Not accessible outside the class.

5. Accessing Sub-Packages:

Sub-packages are not automatically imported; you need to import them explicitly.

Example:

```
import mypackage.subpackage.MySubClass;
```

6. Fully Qualified Name Usage:

Without the `import` statement:

```
public class Main {
    public static void main(String[] args) {
```

```
        mypackage.MyClass obj = new mypackage.MyClass(); // Fully qualified name
        obj.display();
    }
}
```

7. Importance of Accessing Packages:

Allows reuse of code by accessing pre-defined or user-defined packages.

Helps in managing large projects with modularized code.

Simplifies the usage of **built-in packages** like `java.util`, `java.io`.

3.3.1 Types of Error:

Here's a concise explanation of Types of Errors in Java:

1. What are Errors?

Errors are issues in a program that disrupt its normal execution. They can occur due to syntax mistakes, incorrect logic, or system-level issues.

2. Types of Errors in Java:

1. Compile-Time Errors:

Definition:

Errors detected by the compiler during the compilation process.

Causes:

Syntax errors (e.g., missing semicolons, unmatched braces).

Incorrect data types.

Missing imports or undefined variables.

Example:

```
public class Main {
    public static void main(String[] args) {
        int num = "String"; // Compile-time error: incompatible types
    }
}
```

2. Runtime Errors:

Definition:

Errors that occur while the program is running.

Causes:

Invalid user input.

Divide-by-zero errors.

Null pointer dereferences.

Example:

```
public class Main {
    public static void main(String[] args) {
        int num = 10 / 0; // Runtime error: ArithmeticException
    }
}
```

3. Logical Errors:

Definition:

Errors in the program logic that produce incorrect results.

Causes:

Faulty algorithms.

Incorrect conditions in loops or if statements.

Example:

```
public class Main {
    public static void main(String[] args) {
        int num = 5;
        System.out.println("Square is: " + (num + num)); // Logical error:
      should be num * num
    }
}
```

4. Errors vs. Exceptions:

Errors can also refer to **system-level errors** (e.g., `StackOverflowError`, `OutOfMemoryError`) that are part of the `java.lang.Error` class. Unlike exceptions, these errors are not meant to be handled in code.

Example:

```
public class Main {
    public static void main(String[] args) {
        recursiveMethod(); // StackOverflowError
    }

    static void recursiveMethod() {
        recursiveMethod();
    }
}
```

3. Summary Table:

Error Types	When it Occurs	Examples
Compile-Time Errors	During compilation	Syntax errors, missing imports
Runtime Errors	During execution	Divide by zero, null pointers
Logical Errors	Produces wrong output	Incorrect formulas, conditions

4. Importance:

Understanding error types is critical for:

Writing error-free programs.

Debugging effectively.

Improving software reliability.

3.3.2 Multiple catch statement:

Here's a concise explanation of Multiple Catch Statements in Java:

1. What is a Multiple Catch Statement?

In Java, a **multiple catch statement** allows you to handle multiple types of exceptions in a single `try` block. Each `catch` block is used to handle a specific exception type, ensuring precise error handling.

2. Syntax:

```
try {
    // Code that might throw exceptions
} catch (ExceptionType1 e1) {
    // Handle ExceptionType1
} catch (ExceptionType2 e2) {
    // Handle ExceptionType2
} catch (ExceptionType3 e3) {
    // Handle ExceptionType3
}
```

3. Rules for Multiple Catch Blocks:

1. Order of Catch Blocks:

Catch blocks should be ordered from the most specific exception to the most general exception (e.g., `ArithmeticException` before `Exception`), as Java uses the first matching catch block.

2. Unique Handling:

Each catch block handles a specific type of exception.

3. Single Execution:

Only one catch block executes for a thrown exception, even if multiple blocks could potentially match.

4. Example:

```
public class MultipleCatchExample {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            System.out.println(numbers[5]); // ArrayIndexOutOfBoundsException
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception caught!");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Out of Bounds Exception caught!");
        } catch (Exception e) {
            System.out.println("General Exception caught!");
        }
```

```
        }
    }
```

Output:

Array Index Out of Bounds Exception caught!

5. Enhanced Catch Block (Java 7+):

Java 7 introduced the ability to catch multiple exceptions in a single `catch` block using the `|` operator:

```
try {
    int num = Integer.parseInt("abc"); // NumberFormatException
} catch (NumberFormatException | NullPointerException e) {
    System.out.println("Exception caught: " + e);
}
```

6. Advantages of Multiple Catch Statements:

Provides **specific error handling** for different exceptions.

Improves **code readability** and **maintainability**.

Reduces the chance of **missing critical exceptions**.

7. Key Points:

Always ensure more specific exceptions are caught before generic ones.

Proper exception handling ensures program stability even in unexpected scenarios.

3.3.3 Creating User defined Exception:

Here's a concise explanation of Creating User-Defined Exceptions in Java:

1. What is a User-Defined Exception?

A **user-defined exception** is a custom exception class created by a programmer to handle specific application scenarios that are not covered by Java's built-in exceptions. These exceptions are subclasses of the `Exception` class or its descendants.

2. Steps to Create and Use a User-Defined Exception:

Step 1: Define a Custom Exception Class

Extend the `Exception` or `RuntimeException` class.

Provide a constructor to initialize the exception message.

Optionally override methods like `toString()` or `getMessage()`.

Step 2: Throw the Custom Exception

Use the `throw` keyword to throw the custom exception in your code.

Step 3: Handle the Exception

Use a `try-catch` block to handle the exception.

3. Example Program:

Create the Custom Exception

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message); // Pass the message to the Exception class
    }
}
```

Use the Custom Exception

```
public class Main {
    public static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above.");
        } else {
            System.out.println("Access granted.");
        }
    }

    public static void main(String[] args) {
        try {
            checkAge(16); // This will throw the custom exception
        } catch (InvalidAgeException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
    }
}
```

Output:

```
Caught Exception: Age must be 18 or above.
```

4. Advantages of User-Defined Exceptions:

1. Custom Error Messages:

Provide meaningful error messages tailored to specific application requirements.

2. Enhanced Code Clarity:

Clearly defines and separates application-specific errors.

3. Reusability:

Custom exceptions can be reused across multiple parts of an application.

5. Key Points:

Checked Exceptions:

If the custom exception extends `Exception`, it must be declared in the `throws` clause and handled explicitly.

Unchecked Exceptions:

If it extends `RuntimeException`, it does not need to be declared in the `throws` clause.

Creating user-defined exceptions allows developers to design more robust and maintainable applications by clearly handling specific error scenarios.

3.3.4 Finally clause:

Here's a concise explanation of the Finally Clause in Java:

1. What is the Finally Clause?

The **`finally` clause** in Java is a block of code that always executes after a `try` block, regardless of whether an exception was thrown or caught. It is used to perform cleanup operations, such as closing files, releasing resources, or resetting variables.

2. Syntax:

```
try {
    // Code that might throw an exception
} catch (ExceptionType e) {
    // Handle the exception
} finally {
    // Cleanup code
}
```

3. Key Features of Finally Clause:

1. Always Executes:

The `finally` block executes whether an exception is thrown or not.

It also executes if there is a `return` statement in the `try` or `catch` block.

2. Cleanup Operations:

Used for releasing resources like closing files, sockets, or database connections.

3. Does Not Execute in Rare Cases:

When the JVM exits due to a `System.exit()` call.

If the program is terminated abruptly (e.g., power failure).

4. Example Program:

Example 1: With Exception

```
public class FinallyExample {
    public static void main(String[] args) {
        try {
            int num = 10 / 0; // ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e);
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

```
Exception caught: java.lang.ArithmeticException: / by zero
```

Finally block executed.

Example 2: Without Exception

```
public class FinallyExample {
    public static void main(String[] args) {
        try {
            int num = 10 / 2; // No exception
            System.out.println("Result: " + num);
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e);
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

```
Result: 5
Finally block executed.
```

5. Advantages of Finally Clause:

1. Ensures that **critical cleanup code** runs, even if an exception occurs.
2. Makes the code more **reliable** and **robust** by handling resource management.
3. Reduces the risk of **resource leaks** (e.g., unclosed files or connections).

6. Key Points to Remember:

The `finally` block is **optional** but highly recommended for resource management.
It works with `try-catch`, even if the `catch` block is absent.
Ensures proper program termination by executing critical tasks regardless of exceptions.

Unit IV

4. String, Stream and Applet Programming

- **4.1 Introduction and String Classes**
- **4.2 String Buffer Class**
- **4.3 Stream Classes**
  - **4.3.1 Types of Streams**
  - **4.3.2 Byte Stream Classes**
  - **4.3.3 Character Stream Classes**
- **4.4 Introduction and creating Applets**
- **4.5 Applet Life Cycle**
- **4.6 Applet Tag**
- **4.7 Passing Parameters to Applets**

4.1 Introduction and String Classes:

Here's a concise explanation of Introduction and String Classes in Java:

1. Introduction to Strings in Java:

A **String** in Java is a sequence of characters, represented as a **class** in the `java.lang` package.
Strings are **immutable**, meaning their values cannot be changed after creation.
Java provides built-in support for creating, manipulating, and performing operations on strings.

2. Creating Strings:

Strings can be created in two ways:

1. Using String Literals:

```
String str = "Hello";
```

Stored in the **String pool**, which helps save memory.

2. Using the `new` Keyword:

```
String str = new String("Hello");
```

Creates a new object in the heap memory.

3. Common String Classes in Java:

1. String Class:

The `String` class provides methods for manipulating strings.
Example:

```
String str = "Java";
System.out.println(str.length());        // Output: 4
System.out.println(str.toUpperCase());   // Output: JAVA
```

2. StringBuffer Class:

Mutable version of String:

Allows modification of string content without creating new objects.

Thread-Safe:

Suitable for multi-threaded applications.

Example:

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb); // Output: Hello World
```

3. StringBuilder Class:

Similar to `StringBuffer` but **not thread-safe**.

Faster than `StringBuffer` for single-threaded applications.

Example:

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" Java");
System.out.println(sb); // Output: Hello Java
```

4. Key String Methods:

Method	Description	Example
length()	Returns the length of the string.	"Hello".length() → 5
charAt(index)	Returns the character at the index.	"Java".charAt(1) → 'a'
substring(start, end)	Extracts a substring.	"Java".substring(1, 3) → "av"
concat(str)	Concatenates strings.	"Hello".concat(" World") → "Hello World"
equals()	Compares two strings (Case sensitive).	"Java".equals("java") → false
toUpperCase()	Converts to uppercase.	"java".toUpperCase() → "JAVA"

5. Comparison of String, StringBuffer, and StringBuilder:

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (Immutable)	Yes	No
Performance	Slow	Moderate	Fast

6. Importance:

Strings are fundamental for handling text in Java.

Classes like `StringBuffer` and `StringBuilder` provide efficient ways to manipulate strings in different scenarios.

Proper understanding helps in choosing the right class for optimal performance.

4.2 String Buffer Class:

Here's a concise explanation of the StringBuffer Class in Java:

1. What is the StringBuffer Class?

The **`StringBuffer` class** in Java is a part of the `java.lang` package and is used to create mutable (modifiable) strings. Unlike the `String` class, which creates immutable strings, `StringBuffer` allows modifications such as appending, inserting, or deleting characters without creating a new object.

2. Key Features of StringBuffer:

1. **Mutability:** Contents of a `StringBuffer` object can be modified.
2. **Thread-Safe:** It is synchronized, meaning it is safe to use in multi-threaded applications.
3. **Efficient:** Performs better than `String` when frequent modifications are required.

3. Creating a StringBuffer Object:

Syntax:

```
StringBuffer sb = new StringBuffer("Initial String");
```

Constructors:

1. **`StringBuffer()`:**
   Creates an empty buffer with default capacity (16 characters).

2. **`StringBuffer(String str)`:**
   Initializes the buffer with the given string.

3. **`StringBuffer(int capacity)`:**
   Creates a buffer with the specified capacity.

4. Common Methods of StringBuffer:

Method	Description	Example
append(String s)	Adds the specified string to the end.	sb.append(" World"); → "Hello World"
insert(int offset, s)	Inserts the string at a specified index.	sb.insert(5, " Java"); → "Hello Java"
replace(start, end, s)	Replaces characters from start to end.	sb.replace(0, 5, "Hi"); → "Hi Java"
delete(start, end)	Deletes characters in the range.	sb.delete(2, 4); → "Heo"
reverse()	Reverses the string content.	sb.reverse(); → "avaJ olleH"
capacity()	Returns the buffer capacity.	sb.capacity(); → 16 (default capacity)

5. Example Program:

```
public class StringBufferExample {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");
```

```
        // Append
        sb.append(" World");
        System.out.println("After append: " + sb); // Output: Hello World

        // Insert
        sb.insert(6, "Java ");
        System.out.println("After insert: " + sb); // Output: Hello Java World

        // Replace
        sb.replace(6, 10, "Awesome");
        System.out.println("After replace: " + sb); // Output: Hello Awesome World

        // Reverse
        sb.reverse();
        System.out.println("After reverse: " + sb); // Output: dlroW emosewA olleH
    }
}
```

6. Comparison with StringBuilder:

Feature	StringBuffer	StringBuilder
Mutability	Mutable	Mutable
Thread-safe	Yes	No
Performance	Slower	Faster

7. Advantages of StringBuffer:

1. Allows modification of strings without creating new objects.
2. Thread-safe for multi-threaded environments.
3. Useful for frequent string operations, such as in loops or complex concatenations.

8. Key Points:

Use `StringBuffer` when working in a **multi-threaded** application.
For **single-threaded** applications, `StringBuilder` is preferred due to better performance.
Helps optimize memory usage and performance during string manipulation tasks.

4.3.1 Types of Streams:

Here's a concise explanation of Types of Streams in Java:

1. What are Streams in Java?

Streams in Java represent a continuous flow of data between a source (e.g., file, memory) and a destination. Streams are used for input and output operations in Java, and they can handle data in different formats such as bytes or characters.

2. Types of Streams:

Streams in Java are broadly categorized based on two criteria:

1. Based on Direction:

Input Stream:

Used to read data from a source.

Example: Reading data from a file or console.

Classes: `FileInputStream`, `BufferedInputStream`, `InputStreamReader`.

Output Stream:

Used to write data to a destination.

Example: Writing data to a file or console.

Classes: `FileOutputStream`, `BufferedOutputStream`, `OutputStreamWriter`.

2. Based on Data Type:

Byte Streams:

Handles binary data (e.g., images, audio, and video files).

Reads or writes one byte at a time.

Classes:

Input: `FileInputStream`, `BufferedInputStream`.

Output: `FileOutputStream`, `BufferedOutputStream`.

Character Streams:

Handles text data (e.g., `.txt` files, Unicode characters).

Reads or writes one character at a time.

Classes:

Input: `FileReader`, `BufferedReader`.

Output: `FileWriter`, `BufferedWriter`.

3. Key Byte and Character Stream Classes:

Category	Input Stream Classes	Output Stream Classes
Byte Streams	FileInputStream, DataInputStream	FileOutputStream, DataOutputStream
Character Streams	FileReader, BufferedReader	FileWriter, BufferedWriter

4. Example Programs:

Byte Stream Example:

```
import java.io.*;

public class ByteStreamExample {
    public static void main(String[] args) throws IOException {
        FileInputStream fis = new FileInputStream("input.txt");
        int data;
        while ((data = fis.read()) != -1) {
            System.out.print((char) data);
             // Reads byte and converts to character
```

```
            }
            fis.close();
        }
    }
```

Character Stream Example:

```
import java.io.*;

public class CharacterStreamExample {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("input.txt");
        int data;
        while ((data = fr.read()) != -1) {
            System.out.print((char) data);
            // Reads character directly
        }
        fr.close();
    }
}
```

5. Summary Table:

Type	Handles	Examples
Input Stream	Reads data from source	FileInputStream, FileReader
Output Stream	Writes data to destination	FileOutputStream, FileWriter
Byte Stream	Binary data	FileInputStream, FileOutputStream
Character Stream	Text data	FileReader, FileWriter

6. Advantages of Using Streams:

1. Simplifies handling of input/output operations.
2. Allows efficient reading/writing of both text and binary data.
3. Provides a flexible mechanism to work with different data sources like files, memory, or network.

Understanding these types helps developers choose the right stream based on the type of data and the operation.

4.3.2 Byte Stream Classes:

Here's a concise explanation of Byte Stream Classes in Java:

1. What are Byte Stream Classes?

Byte Stream classes in Java are used to perform input and output operations on **binary data**, such as images, audio, video, or any non-text files. They read or write data **one byte at a time** (8 bits).

Byte streams are part of the `java.io` package.

They are the foundation for low-level file I/O operations.

2. Types of Byte Stream Classes:

1. Input Stream Classes:

Used to read bytes from a source.

Key Classes:

`FileInputStream`:

Reads bytes from a file.

`BufferedInputStream`:

Buffers input for efficient reading.

`DataInputStream`:

Reads primitive data types (e.g., `int`, `float`) as bytes.

`ObjectInputStream`:

Reads objects from a file.

2. Output Stream Classes:

Used to write bytes to a destination.

Key Classes:

`FileOutputStream`:

Writes bytes to a file.

`BufferedOutputStream`:

Buffers output for efficient writing.

`DataOutputStream`:

Writes primitive data types as bytes.

`ObjectOutputStream`:

Writes objects to a file.

3. Important Methods in Byte Stream Classes:

Method	Description
read()	Reads a single byte of data from the input stream.
read(byte[] b)	Reads multiple bytes into a byte array.
write(int b)	Writes a single byte to the output stream.
write(byte[] b)	Writes multiple bytes from a byte array.
close()	Closes the stream to release resources.

4. Example Programs:

FileInputStream Example:

```
import java.io.*;

public class ByteInputExample {
    public static void main(String[] args) {
        try (FileInputStream fis = new FileInputStream("input.txt")) {
            int data;
            while ((data = fis.read()) != -1) {
                System.out.print((char) data);
```

```
                                        // Reads and prints each byte as a character
                                }
                        } catch (IOException e) {
                                e.printStackTrace();
                        }
                }
        }
```

FileOutputStream Example:

```
import java.io.*;

public class ByteOutputExample {
        public static void main(String[] args) {
                try (FileOutputStream fos = new FileOutputStream("output.txt")) {
                        String content = "Hello, Java!";
                        fos.write(content.getBytes());
                        // Converts string to bytes and writes to the file
                        System.out.println("Data written successfully.");
                } catch (IOException e) {
                        e.printStackTrace();
                }
        }
}
```

5. Advantages of Byte Stream Classes:

1. Suitable for working with **binary data** (e.g., images, audio, video).
2. Provides flexibility to handle low-level I/O operations.
3. Compatible with a wide range of data sources and destinations, like files, sockets, or memory.

6. Key Points to Remember:

Use `FileInputStream` and `FileOutputStream` for file-related byte I/O.

Buffered streams (`BufferedInputStream` and `BufferedOutputStream`) improve performance by reducing disk I/O.

Always close streams after use to release system resources.

By using Byte Stream classes, Java developers can handle binary data efficiently in applications.

4.3.3 Character Stream Classes:

Here's a concise explanation of Character Stream Classes in Java:

1. What are Character Stream Classes?

Character Stream classes in Java are designed to handle **text data** (character data). Unlike byte streams that work with raw binary data, character streams work with **16-bit Unicode characters**. This makes them ideal for reading and writing text files.

Character streams are part of the `java.io` package.

They read or write data **one character at a time**.

2. Types of Character Stream Classes:

1. Reader Classes (Input):

Used to read characters from a source.

Key Classes:

`FileReader`:

Reads characters from a file.

`BufferedReader`:

Buffers input to improve performance and provides methods like `readLine()`.

`InputStreamReader`:

Converts byte streams to character streams.

`CharArrayReader`:

Reads characters from a character array.

2. Writer Classes (Output):

Used to write characters to a destination.

Key Classes:

`FileWriter`:

Writes characters to a file.

`BufferedWriter`:

Buffers output for efficient writing and provides methods like `newLine()`.

`OutputStreamWriter`:

Converts character streams to byte streams.

`CharArrayWriter`:

Writes characters to a character array.

3. Common Methods in Character Stream Classes:

Method	Description
read()	Reads a single character.
read(char[] cbuf)	Reads characters into a character array.
write(int c)	Writes a single character.
write(char[] cbuf)	Writes characters from a character array.
close()	Closes the stream and releases system resources.

4. Example Programs:

FileReader Example:

```
import java.io.*;

public class CharacterInputExample {
    public static void main(String[] args) {
        try (FileReader fr = new FileReader("input.txt")) {
            int data;
            while ((data = fr.read()) != -1) {
```

```
                    System.out.print((char) data);
                    // Reads and prints each character
                }
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
```

FileWriter Example:

```
import java.io.*;

public class CharacterOutputExample {
    public static void main(String[] args) {
        try (FileWriter fw = new FileWriter("output.txt")) {
            String content = "Hello, Java!";
            fw.write(content);
            // Writes the string as characters to the file
            System.out.println("Data written successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

5. Advantages of Character Stream Classes:

1. Designed for **text-based data**, making them suitable for working with plain text files.
2. Handles **Unicode characters**, ensuring compatibility with global languages.
3. Provides high-level methods (e.g., `readLine()`, `newLine()`) for efficient text handling.

6. Comparison of Key Character Stream Classes:

Class	Description
`FileReader`	Reads text from a file character by character.
`BufferedReader`	Improves reading efficiency and allows reading entire lines.
`FileWriter`	Writes text to a file character by character.
`BufferedWriter`	Improves writing efficiency and allows adding new lines.

7. Key Points to Remember:

Use `BufferedReader` and `BufferedWriter` for better performance with large text files.

Always close streams after use to avoid resource leaks.

Character streams are best suited for working with **text files**, while byte streams are better for **binary data**.

By using character streams, Java developers can efficiently handle text-based operations in their applications.

4.4 Introduction and creating Applets:

Introduction to Applets:

Applets are **small Java programs** that run inside a web browser or an applet viewer. They are part of the **`java.applet`** package and are designed to create dynamic and interactive web applications.

Key Features of Applets:

1. **Embedded in HTML:**
   Applets are embedded in HTML documents using the `<applet>` tag (or `<object>` in modern browsers).

2. **Run in a Sandbox:**
   Applets operate in a restricted environment, ensuring security.

3. **Graphical User Interface (GUI):**
   Applets can create GUI components for user interaction.

4. **Dependent on JVM:**
   They require a Java Virtual Machine (JVM) for execution, usually provided by the browser or a standalone viewer.

Life Cycle of an Applet:

1. **`init()`:**
   Called when the applet is first loaded.

2. **`start()`:**
   Invoked when the applet becomes active.

3. **`paint(Graphics g)`:**
   Used to draw graphics on the applet window.

4. **`stop()`:**
   Called when the applet is stopped temporarily (e.g., browser tab is minimized).

5. **`destroy()`:**
   Invoked when the applet is permanently terminated.

Steps to Create an Applet:

1. **Import Necessary Packages:**
   Import `java.applet.Applet` and `java.awt.Graphics`.

2. **Extend the Applet Class:**
   Your class should extend the `Applet` class to inherit its functionalities.

3. **Override Life Cycle Methods:**

At a minimum, override the `paint(Graphics g)` method to define the applet's behavior.

4. Compile the Applet Code:

Use `javac` to compile the Java program.

5. Run the Applet:

Use an applet viewer or embed it in an HTML file for execution.

Example: Creating a Simple Applet:

Java Code:

```
import java.applet.Applet;
import java.awt.Graphics;

// Extend the Applet class
public class HelloApplet extends Applet {
    // Overriding the paint method to display content
    public void paint(Graphics g) {
        g.drawString("Hello, Applet World!", 50, 50); // Draws text on the screen
    }
}
```

HTML File:

```
<html>
<body>
    <applet code="HelloApplet.class" width="300" height="200"></applet>
</body>
</html>
```

Running the Applet:

1. Compile the Code:

`javac HelloApplet.java`

2. Run Using Applet Viewer:

appletviewer HelloApplet.html

Advantages of Applets:

1. Enables **dynamic web content**.
2. Provides a platform for **interactive applications**.
3. Runs securely in a **sandbox** environment, preventing unauthorized access.

Limitations of Applets:

1. Modern browsers have **deprecated applet support** due to security concerns.
2. Requires a JVM plugin, making it platform-dependent.
3. Limited use in modern web development due to alternatives like JavaScript and frameworks.

This explanation highlights both the creation and basic functioning of applets, providing a complete understanding.

4.5 Applet Life Cycle:

Applet Life Cycle:

An applet is a small Java program that runs in a web browser or applet viewer. Its life cycle is defined by a set of methods that the Java runtime environment invokes during the applet's execution.

Stages of Applet Life Cycle:

1. Initialization (`init()`):

This method is called **once** when the applet is loaded for the first time.

It is used to initialize variables, set up user interface components, and allocate resources.

Syntax:

```
public void init() {
    // Initialization code
}
```

2. Starting (`start()`):

This method is called each time the applet becomes active or visible (e.g., when the browser is opened or switched back to the applet tab).

It contains the code to start animations, threads, or other processes.

Syntax:

```
public void start() {
    // Start or resume applet execution
}
```

3. Painting (`paint(Graphics g)`):

This method is invoked whenever the applet needs to redraw its content, such as during initialization or when the window is resized.

It is responsible for rendering graphics and displaying content.

Syntax:

```
public void paint(Graphics g) {
    g.drawString("Hello Applet!", 50, 50);
}
```

4. Stopping (`stop()`):

This method is called when the applet is temporarily stopped (e.g., when the user navigates away from the web page or minimizes the browser).

It is used to pause animations, stop threads, or release temporary resources.

Syntax:

```
public void stop() {
    // Pause the applet
}
```

5. Destruction (`destroy()`):

This method is called **once** when the applet is being removed permanently, such as when the browser is closed.

It is used to clean up resources, such as closing file streams or stopping threads.

Syntax:

```
public void destroy() {
    // Cleanup code
}
```

Applet Life Cycle Methods Flow:

1. **`init()`** → Called when the applet starts for the first time.

2. **`start()`** → Called after `init()` and every time the applet becomes active.

3. **`paint(Graphics g)`** → Called when the applet is displayed or needs to refresh its content.

4. **`stop()`** → Called when the applet is minimized or inactive.

5. **`destroy()`** → Called when the applet is terminated.

Diagram of Applet Life Cycle:

```
Browser/Applet Viewer Event
            ↓
            init()
            ↓
            start()
            ↓
            paint()
            ↓
stop()   (on browser/tab close)
            ↓
destroy() (on applet termination)
```

Example Code:

```
import java.applet.Applet;
import java.awt.Graphics;

public class LifeCycleApplet extends Applet {
    public void init() {
        System.out.println("Applet initialized");
    }

    public void start() {
        System.out.println("Applet started");
    }

    public void paint(Graphics g) {
        g.drawString("Applet Life Cycle!", 50, 50);
```

```
        }

        public void stop() {
            System.out.println("Applet stopped");
        }

        public void destroy() {
            System.out.println("Applet destroyed");
        }
    }
```

Key Points:

1. **`init()` and `destroy()`** are called only once.
2. **`start()` and `stop()`** are called multiple times during an applet's lifetime.
3. Applets use the `paint()` method for graphical updates.

This systematic flow ensures smooth execution of applet programs.

4.6 Applet Tag:

Applet Tag in HTML:

The **`<applet>`** tag in HTML is used to embed a Java applet into a web page. This tag allows the applet to run within a browser, provided the browser has Java support enabled.

Syntax of `<applet>` Tag:

```
<applet code="ClassName.class" width="width_value" height="height_value">
    <!-- Optional parameters can be added here -->
</applet>
```

Attributes of `<applet>` Tag:

1. `code`:

Specifies the name of the Java applet class file (e.g., `MyApplet.class`).
This is a mandatory attribute.
Example:

```
<applet code="MyApplet.class"></applet>
```

2. `width`:

Specifies the width of the applet's display area (in pixels).
This is a mandatory attribute.
Example:

```
<applet width="300"></applet>
```

3. `height`:

Specifies the height of the applet's display area (in pixels).
This is a mandatory attribute.
Example:

```
<applet height="200"></applet>
```

4. `alt` (Optional):

Provides alternative text to display if the browser does not support applets.

5. `codebase` (Optional):

Specifies the base URL or directory where the applet's class files are located.

Example:

```
<applet codebase="https://example.com/applets/" code="MyApplet.class"></applet>
```

6. `name` (Optional):

Assigns a name to the applet instance, allowing it to be referenced by scripts.

7. `align` (Optional):

Specifies the alignment of the applet within the web page.

8. `archive` (Optional):

Specifies the location of a JAR file containing the applet's class files for faster loading.

Example:

```
<applet code="MyApplet.class" archive="myapplet.jar"></applet>
```

Example of `<applet>` Tag:

HTML Code:

```
<html>
<body>
    <h1>Welcome to Applets</h1>
    <applet code="HelloApplet.class" width="300" height="200">
        Your browser does not support applets.
    </applet>
</body>
</html>
```

Java Applet Code (HelloApplet.java):

```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello, Applet!", 50, 50);
    }
}
```

Key Points:

1. Mandatory Attributes:

`code`, `width`, and `height` are required for the `<applet>` tag to function.

2. Compatibility:

Modern browsers no longer support applets due to security concerns, but the `<applet>` tag can still be used with tools like the applet viewer.

3. Alternative Tags:

The `<object>` or `<embed>` tags are used in newer standards to include Java applications.

Deprecated Status:

The `<applet>` tag is **deprecated** in HTML5, and most modern browsers have discontinued support for Java applets. However, applet programs can still be executed using the Java applet viewer or older browser configurations.

This explanation provides a complete understanding of the `<applet>` tag and its usage.

4.7 Passing Parameters to Applets:

Passing Parameters to Applets:

In Java, parameters can be passed to an applet through the HTML file using the `<param>` tag. These parameters are used to customize the behavior of the applet dynamically based on the values provided in the HTML file.

Steps to Pass Parameters to Applets:

1. Define Parameters in HTML:

Use the `<param>` tag inside the `<applet>` tag to define parameters.

Each `<param>` tag has two attributes:

`name`: The name of the parameter.

`value`: The value assigned to the parameter.

2. Access Parameters in Java Applet:

In the applet code, use the **`getParameter(String name)`** method to retrieve the parameter values.

Syntax of `<param>` Tag:

```
<applet code="ClassName.class" width="width_value" height="height_value">
    <param name="parameter_name" value="parameter_value">
</applet>
```

Example: Passing Parameters to an Applet:

HTML Code:

```
<html>
<body>
    <h1>Parameter Passing Example</h1>
    <applet code="ParamApplet.class" width="300" height="200">
        <param name="message" value="Hello, Applet!">
        <param name="fontSize" value="20">
    </applet>
</body>
```

```
</html>
```

Java Applet Code (ParamApplet.java):

```
import java.applet.Applet;
import java.awt.*;

public class ParamApplet extends Applet {
    String message;
    int fontSize;

    public void init() {
        // Retrieving parameters
        message = getParameter("message");
        if (message == null) {
            message = "Default Message"; // Fallback if no parameter is passed
        }

        String fontSizeParam = getParameter("fontSize");
        if (fontSizeParam != null) {
            fontSize = Integer.parseInt(fontSizeParam);
        } else {
            fontSize = 12; // Default font size
        }
    }

    public void paint(Graphics g) {
        g.setFont(new Font("Arial", Font.PLAIN, fontSize));
        g.drawString(message, 50, 100); // Display the parameter message
    }
}
```

Explanation of the Code:

1. HTML File:

The `<param>` tags define two parameters:

`message`: A string to display.

`fontSize`: The size of the text.

2. Java Applet:

The `getParameter("name")` method retrieves the parameter values from the HTML file.

The applet displays the message with the specified font size.

Key Points to Remember:

1. Default Values:

Always provide a fallback value in case the parameter is not passed.

2. Type Conversion:

Parameters retrieved via `getParameter` are strings. Convert them to other data types if necessary (e.g., `Integer.parseInt()` for integers).

3. Dynamic Behavior:

Using parameters allows applets to behave differently based on the values provided in the HTML file.

Advantages of Passing Parameters:

1. Customization:

Applets can be dynamically customized without modifying the Java code.

2. Flexibility:

Parameters allow applets to adapt to different use cases on the same web page.

This explanation covers the concept, implementation, and advantages of passing parameters to applets.